Boundary Limited Observer Systems

Computational and Formal Supplement

Christopher Ezernack

The finite model assigns hidden labels to each source state and records only the base coordinate. The code enumerates states with identical readings, calculates the possible results of each processing rule, and compares interfaces. The tables identify the corresponding Lean declarations (Appendix A), finite counts (Appendix B), and source files (Appendix D).

1 From source states to the reported counts

The code for the finite calculations is in `verification/reference_code/simulations/boundary_observer/`. The reference implementation is commit `eb96b984ae999ee6cbc0aae33c2d501c46d9540f`.

1.1 Construct the hidden states

Every base state receives the same number of labels. The observation map reads only the base coordinate.

```
    latent_states = tuple(
        (state, hidden)
        for state in base.latent_states
        for hidden in range(multiplicity)
    )
    return GenerativeModel(
        name=name or f"{base.name}_{multiplicity}_fold_extension",
        latent_states=latent_states,
        observation_space=base.observation_space,
        observation_by_state={
            state: base.observe(state[0]) for state in latent_states
        },
        hidden_factor_kind="finite",
        hidden_multiplicity=multiplicity,
        base_model_name=base.name,
    )
```

1.2 Count pairs and process the reachable outputs

A fiber of size k contributes $\binom{k}{2}$ unordered pairs. The runner sums these contributions over all fibers.

```
        return sum(
            comb(len(fiber), 2)
            for fiber in self.observation_fibers().values()
        )
```

A processing rule acts once on each reachable observation. The set of results combines repeated outcomes.

```
    def possible_outcomes(self, model: GenerativeModel) -> frozenset[Hashable]:
        return frozenset(
            self.transform(observation)
            for observation in model.reachable_observations
        )
```

1.3 Compare models and interfaces

The model grouping uses the reachable set as its key. Two models with that key occur in the same empirical class even if their latent spaces differ.

```
    grouped: dict[frozenset[Hashable], list[GenerativeModel]] = {}
    for model in models:
        grouped.setdefault(model.reachable_observations, []).append(model)
    classes = [
        EmpiricalEquivalenceClass(reachable, tuple(members))
        for reachable, members in grouped.items()
    ]
```

The refinement calculation enumerates each unordered pair of world states once. It records a pair when the fine interface distinguishes it and the coarse interface identifies it.

```
        return tuple(
            (left, right)
            for left, right in combinations(self.fine.world_states, 2)
            if not self.fine.observationally_equivalent(left, right)
            and self.coarse.observationally_equivalent(left, right)
        )
```

A Lean declarations

Table 1 identifies the declarations in `BoundaryObserver.lean` for each mathematical result.

Table 1. Definitions and theorems in Lean

Declaration	Paper section	Verified role
`ObserverInterface`	Article, Appendix B	Declares an observation type and a map from world states to observations.
`ObsEq`	Article, Appendix B	Defines state equivalence by equality of accessible observations.
`factorized_quantity_respects_obsEq`	Article, Appendix B	Proves that interface factorized quantities are constant on observation fibers.
`exact_decoder_implies_injective`	Article, Appendix B	Proves that a globally exact decoder implies observation injectivity.
`no_exact_decoder_of_not_injective`	Article, Appendix B	Gives the reconstruction obstruction for a noninjective map.
`internal_test_cannot_distinguish_obsEq`	Article, Appendix B	Proves that an observation only binary test cannot separate one fiber.
`finite_interface_has_collision`	Article, Appendix B	Uses finite cardinality to produce distinct states with one observation.
`finite_interface_has_no_exact_decoder`	Article, Appendix B	Connects finite capacity deficit to exact decoder failure.
`ObservableQuotient`	Article, Appendix C	Quotients world states by observational equivalence.
`observableQuotient_universal_property`	Article, Appendix C	Characterizes unique factorization of fiber constant quantities.
`observable_variation_prevents_factorization`	Article, Appendix C	Shows that variation within one fiber prevents interface factorization.
`InterfaceRefines`	Article, Appendix D	Defines coarsening by a commuting observation equation.
`refinement_preserves_obsEq`	Article, Appendix D	Proves that fine equivalence implies coarse equivalence.
`factorization_monotone_under_refinement`	Article, Appendix D	Lifts every coarse accessible quantity to the fine interface.
`quotientMapOfRefinement_surjective`	Article, Appendix D	Proves that the induced fine to coarse quotient map is surjective.
`strictlyRefines_prevents_reverse_refinement`	Article, Appendix D	Proves that strict refinement cannot be reversed.
`strictlyRefines_quotientMap_not_injective`	Article, Appendix D	Proves that strict coarsening collapses quotient classes.
`quotientMapOfMutualRefinement_bijective`	Article, Appendix D	Establishes quotient bijection under mutual refinement.
`GenerativeModel`	Article, Appendix E	Declares a latent state type and a latent to observation map.
`Produces`	Article, Appendix E	Defines reachability of one observation.
`ModelObsEq`	Article, Appendix E	Defines model equivalence by equality of produced observations.

Declaration	Paper section	Verified role
`ExperimentOutcomeSet`	Article, Appendix E	Collects the possible results of one admissible experiment.
`modelObsEq_preserves_experimentOutcomeSet`	Article, Appendix E	Proves outcome set equality for empirically equivalent models.
`identityExperiment`	Article, Appendix E	Returns the accessible observation unchanged.
`exactLatentDecoder_not_determinedBy_identityExperiment`	Article, Appendix F	Proves that even complete accessible observations do not determine decoder existence.
`PredicateProfile`	Article, Appendix F	Records which accessible predicates can succeed.
`modelObsEq_iff_predicateProfile_eq`	Article, Appendix F	Characterizes empirical equivalence by complete predicate profiles.
`exactLatentDecoder_not_determinedByPredicateProfile`	Article, Appendix F	Shows that complete predicate profiles do not determine exact reconstruction.
`LatentCover`	Article, Appendix G	Declares a surjective observation preserving projection to a base model.
`StrictLatentCover`	Article, Appendix G	Adds noninjectivity of the latent projection.
`hiddenExtensionModel`	Article, Appendix G	Adjoins a hidden factor and ignores it during observation.
`finiteMultiplicityModel`	Article, Appendix G	Creates arbitrary finite hidden multiplicity.
`countableMultiplicityModel`	Article, Appendix G	Creates a natural number hidden coordinate.
`countableMultiplicityFiber_contains_nat`	Article, Appendix G	Embeds the natural numbers into one observation fiber.
`no_inhabited_model_is_latentlyMaximal`	Article, Appendix G	Constructs a strict empirically equivalent extension of every inhabited model.
`EmpiricalModelQuotient`	Article, Appendix H	Quotients generative models by empirical equivalence.
`observationInvariant_iff_factorsThroughEmpiricalModelQuotient`	Article, Appendix H	Characterizes observation invariant properties by quotient factorization.
`empiricalModelQuotient_map_universal_property`	Article, Appendix H	Gives unique descent of arbitrary observation invariant measurements.
`exactLatentDecoder_not_factorsThroughEmpiricalModelQuotient`	Article, Appendix H	Proves that decoder existence is not a property of the empirical class.
`empiricalModelQuotientEquivRealizableObservationSet`	Article, Appendix H	Identifies empirical classes with realizable reachable observation sets.
`boundaryLimitedObserver_synthesis`	Article, Appendix I	Constructs the strict Boolean extension with the five conclusions of the synthesis theorem.

B Finite model data

The files in `data/original_csv/` contain the complete finite calculations. The tables report the values in these files.

B.1 Complete model summary

Table 2. Complete `model_summary.csv`

Model	Latent	Reachable	Avg fiber	Max fiber	Collisions	Multiplicity	Injective	Decoder	Hidden kind
injective_base	6	6	1.00	1	0	1	True	True	none
noninjective_base	6	3	2.00	2	3	1	False	False	none
boolean_hidden_extension	12	6	2.00	2	6	2	False	False	boolean
injective_base_1_fold_extension	6	6	1.00	1	0	1	True	True	finite
injective_base_2_fold_extension	12	6	2.00	2	6	2	False	False	finite
injective_base_3_fold_extension	18	6	3.00	3	18	3	False	False	finite
injective_base_5_fold_extension	30	6	5.00	5	60	5	False	False	finite
injective_base_8_fold_extension	48	6	8.00	8	168	8	False	False	finite

B.2 Complete experiment outcome records

Table 3. Complete `experiment_outcomes.csv`

Experiment	Model	Multiplicity	Possible outcomes	Count	Matches base
identity	injective_base	1	{0, 1, 2, 3, 4, 5}	6	True
identity	boolean_hidden_extension	2	{0, 1, 2, 3, 4, 5}	6	True
identity	injective_base_1_fold_extension	1	{0, 1, 2, 3, 4, 5}	6	True
identity	injective_base_2_fold_extension	2	{0, 1, 2, 3, 4, 5}	6	True
identity	injective_base_3_fold_extension	3	{0, 1, 2, 3, 4, 5}	6	True
identity	injective_base_5_fold_extension	5	{0, 1, 2, 3, 4, 5}	6	True
identity	injective_base_8_fold_extension	8	{0, 1, 2, 3, 4, 5}	6	True
threshold_at_3	injective_base	1	{False, True}	2	True
threshold_at_3	boolean_hidden_extension	2	{False, True}	2	True
threshold_at_3	injective_base_1_fold_extension	1	{False, True}	2	True
threshold_at_3	injective_base_2_fold_extension	2	{False, True}	2	True
threshold_at_3	injective_base_3_fold_extension	3	{False, True}	2	True
threshold_at_3	injective_base_5_fold_extension	5	{False, True}	2	True
threshold_at_3	injective_base_8_fold_extension	8	{False, True}	2	True
categorical_partition	injective_base	1	{'high', 'low', 'middle'}	3	True
categorical_partition	boolean_hidden_extension	2	{'high', 'low', 'middle'}	3	True
categorical_partition	injective_base_1_fold_extension	1	{'high', 'low', 'middle'}	3	True
categorical_partition	injective_base_2_fold_extension	2	{'high', 'low', 'middle'}	3	True
categorical_partition	injective_base_3_fold_extension	3	{'high', 'low', 'middle'}	3	True
categorical_partition	injective_base_5_fold_extension	5	{'high', 'low', 'middle'}	3	True
categorical_partition	injective_base_8_fold_extension	8	{'high', 'low', 'middle'}	3	True
compressed_width_2	injective_base	1	{0, 1, 2}	3	True
compressed_width_2	boolean_hidden_extension	2	{0, 1, 2}	3	True
compressed_width_2	injective_base_1_fold_extension	1	{0, 1, 2}	3	True
compressed_width_2	injective_base_2_fold_extension	2	{0, 1, 2}	3	True
compressed_width_2	injective_base_3_fold_extension	3	{0, 1, 2}	3	True
compressed_width_2	injective_base_5_fold_extension	5	{0, 1, 2}	3	True
compressed_width_2	injective_base_8_fold_extension	8	{0, 1, 2}	3	True

B.3 Predicate records

Each of three predicates was evaluated on all seven models. The 21 success profiles and the 21 truth tables match those of the base. The table gives the three distinct truth tables.

Table 4. Unique predicate truth tables from `predicate_truth_profiles.csv`

Predicate	Truth table	Success in base	All seven models match
is_even	0:True; 1:False; 2:True; 3:False; 4:True; 5:False	True	True
at_least_three	0:False; 1:False; 2:False; 3:True; 4:True; 5:True	True	True
is_endpoint	0:True; 1:False; 2:False; 3:False; 4:False; 5:True	True	True

B.4 Empirical quotient and varying properties

Table 5. Complete `empirical_quotient.csv`

Class	Reachable set	Latent cardinalities	Models	Multiple cardinalities
1	{0, 1, 2, 3, 4, 5}	6; 12; 18; 30; 48	injective_base; boolean_hidden_extension; injective_base_1_fold_extension; injective_base_2_fold_extension; injective_base_3_fold_extension; injective_base_5_fold_extension; injective_base_8_fold_extension	True
2	{0, 1, 2}	6	noninjective_base	False

Table 6. Complete `nonidentifiable_properties.csv`

Property	Base	Extension	Varies	Reach equal	Outcomes equal
latent cardinality	6	12	True	True	True
maximum observation fiber size	1	2	True	True	True
existence of an exact latent decoder	True	False	True	True	True
presence of a Boolean hidden factor	False	True	True	True	True

B.5 Complete interface and refinement summaries

Table 7. Complete `interface_summary.csv`

Interface	World	Codomain	Reachable	Reachable values	Quotient	Injective
identity_interface	8	8	8	{0, 1, 2, 3, 4, 5, 6, 7}	8	True
paired_bucket_interface	8	4	4	{0, 1, 2, 3}	4	False
binary_region_interface	8	2	2	{'lower', 'upper'}	2	False
relabeled_identity_interface	8	8	8	{'state_0', 'state_1', 'state_2', 'state_3', 'state_4', 'state_5', 'state_6', 'state_7'}	8	True

Table 8. Selected complete fields from `refinement_summary.csv`

Refinement	Fine	Coarse	Fine Q	Coarse Q	Collapsed	Strict	Reverse	Surjective	Injective	Bijective
identity_to_paired	identity_interface	paired_bucket_interface	8	4	4	True	False	True	False	False
paired_to_region	paired_bucket_interface	binary_region_interface	4	2	8	True	False	True	False	False
identity_to_region	identity_interface	binary_region_interface	8	2	12	True	False	True	False	False
identity_to_relabeled	identity_interface	relabeled_identity_interface	8	8	0	False	True	True	True	True
relabeled_to_identity	relabeled_identity_interface	identity_interface	8	8	0	False	True	True	True	True

The files `observation_fibers.csv`, `quotient_maps.csv`, and `collapsed_pairs.csv` contain the unaggregated fiber, class map, and pair records. Each row identifies an individual fiber, class mapping, or pair.

C Finite model figures

The six figures use the CSV data in `data/original_csv/`. The original image files are in `original_repo_pngs/`.

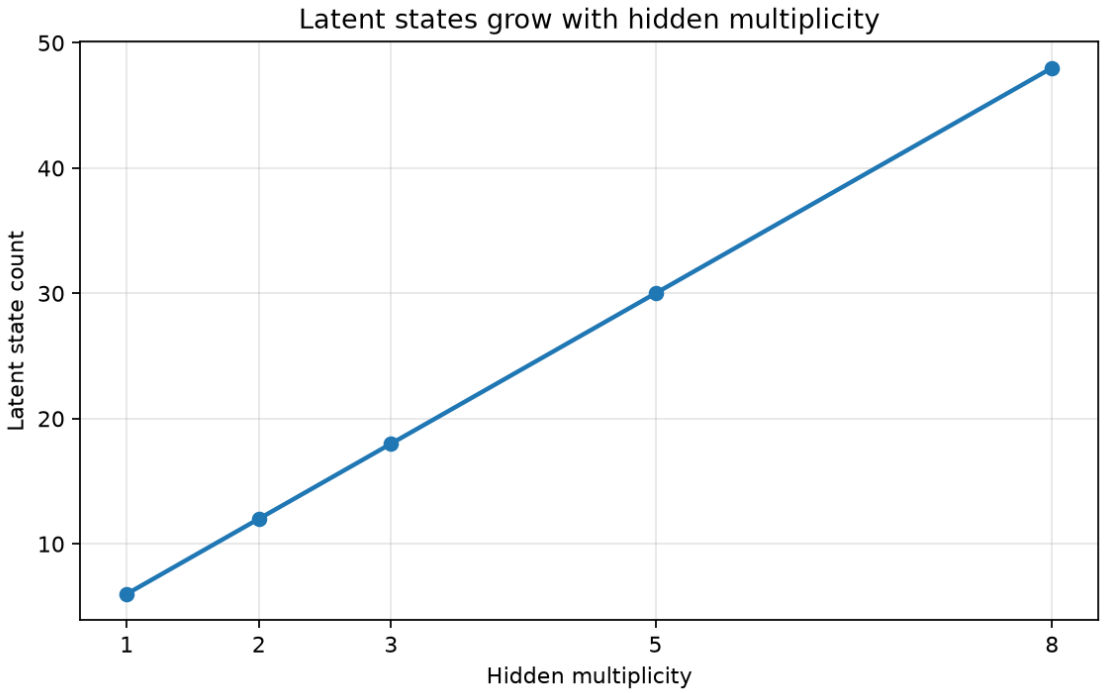


Figure 1. Each hidden label adds six source states.

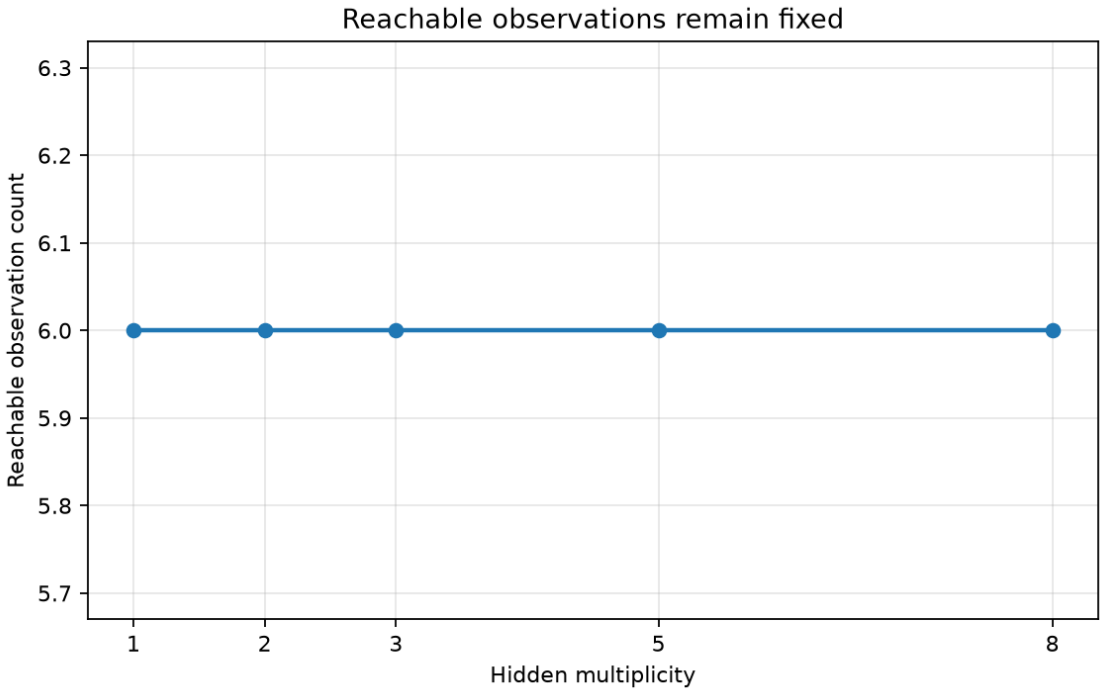


Figure 2. The number of reachable observations stays at six as hidden multiplicity increases.

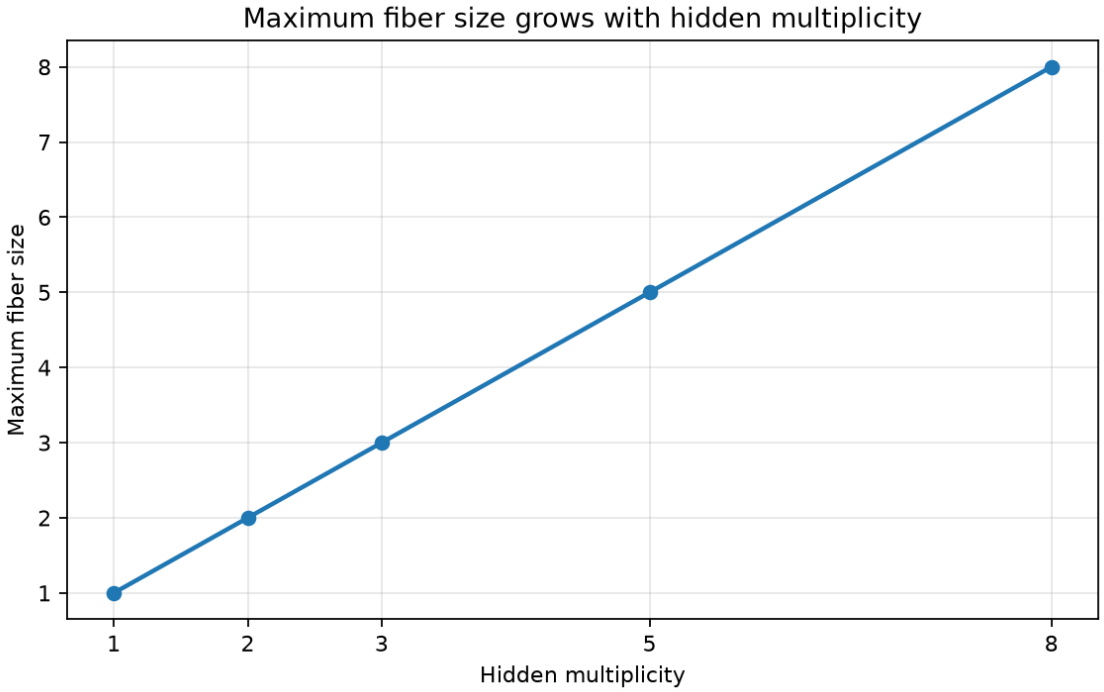


Figure 3. The maximum fiber size equals the number of hidden labels at each multiplicity.

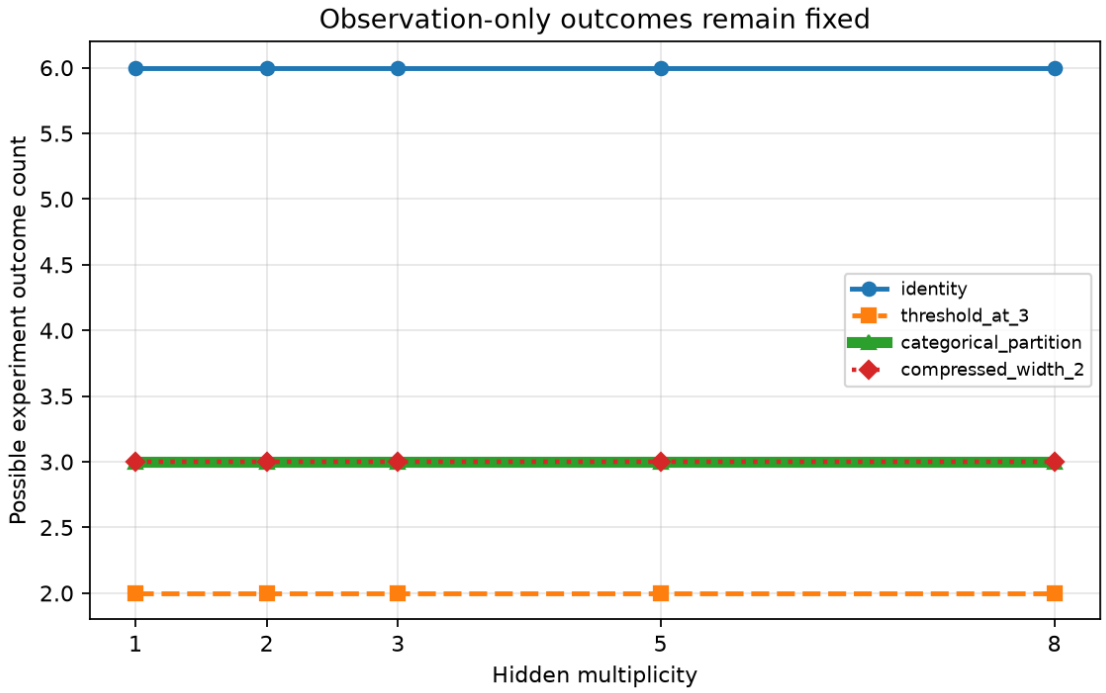


Figure 4. Each rule has a fixed number of possible outcomes across the six-output models.

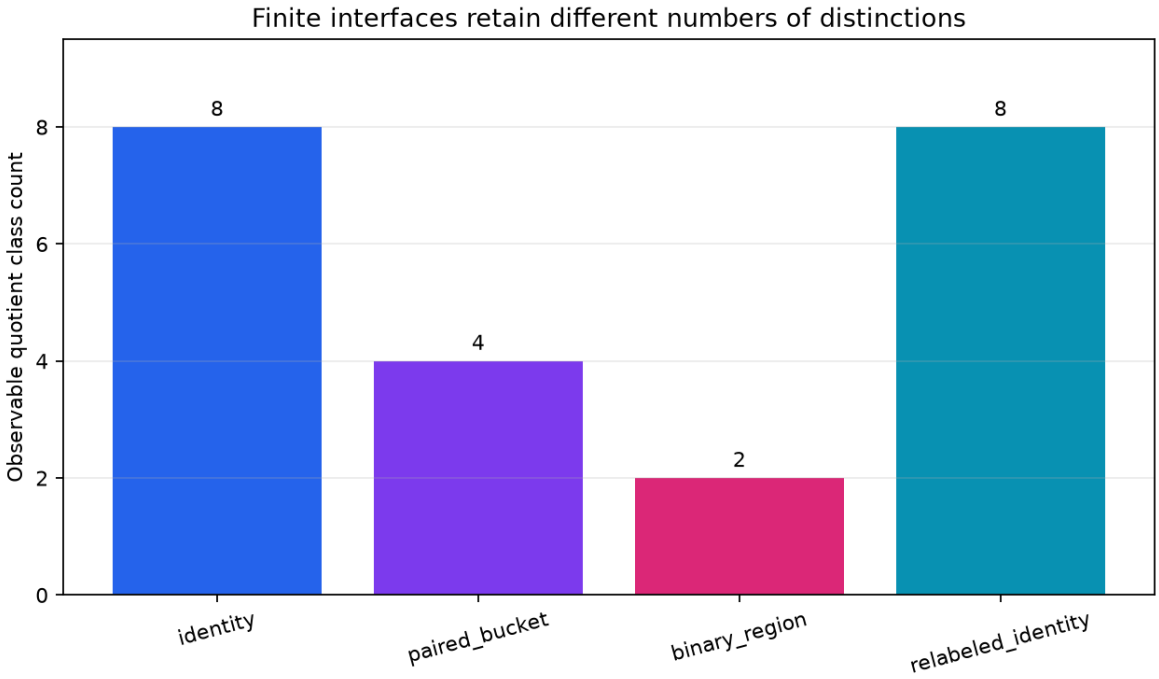


Figure 5. Identity and relabeled identity retain eight classes. Paired buckets retain four and binary regions retain two.

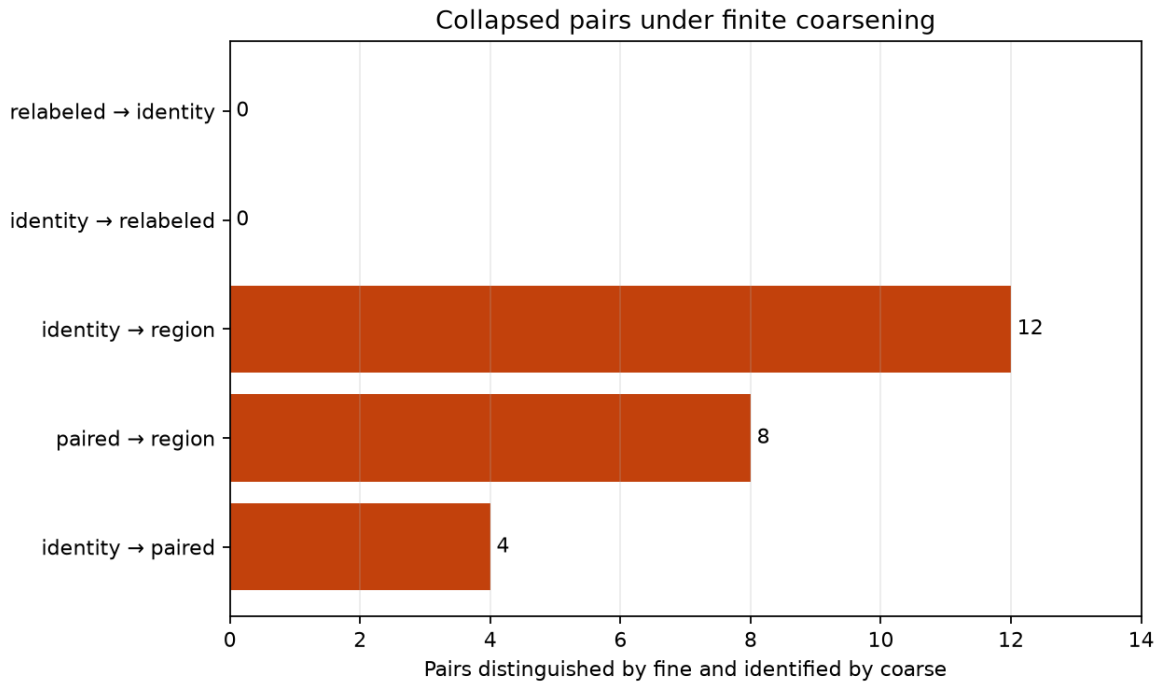


Figure 6. The three strict coarsenings merge four, eight, and twelve pairs, respectively. Relabeling merges none.

D Data provenance and file integrity

The file `data_manifest.json` records the SHA256 digest and Git blob hash of each CSV file. Table 9 gives the row count and the first twelve characters of the Git hash.

Table 9. Committed CSV provenance

File	Data rows	Git blob SHA prefix	Role
collapsed_pairs.csv	24	`fa8eff261917`	Unaggregated strict collapse records
empirical_quotient.csv	2	`f3b32c8e14b1`	Finite empirical classes
experiment_outcomes.csv	28	`7aab2b56cd54`	Possible outcome sets
interface_summary.csv	4	`421487d5c9ce`	Finite interface summaries
model_summary.csv	8	`754a77f53d49`	All finite model summaries
multiplicity_summary.csv	5	`50b3754b844c`	Hidden multiplicity family
nonidentifiable_properties.csv	4	`fb0af96c7411`	Latent property variation
observation_fibers.csv	45	`e7708885ecc8`	Unaggregated observation fibers
predicate_success_profiles.csv	21	`b01193699afb`	Existential predicate profiles
predicate_truth_profiles.csv	21	`bf36618ee015`	Complete finite truth profiles
quotient_maps.csv	36	`498dacd13751`	Unaggregated quotient map entries
refinement_summary.csv	5	`b9cfae935abf`	Refinement and quotient map summary

The manifest also records hashes for the six original PNG figures. The vector figures use the same CSV records. The collision plot uses the pair counts from those records.

D.1 Validation record

Table 10. Formal and computational checks

Check	Recorded result
Repository Lean CI	Passed at commit `ec1ad1d`. `https://github.com/ezernackchristopher97-cloud/BoundaryObserver/actions/runs/42137133326`
Unfinished proof commands	None in project source
Simulation runner	Completed all Boolean, multiplicity, experiment, predicate, quotient, and refinement assertions
Automated tests	18 reference tests and 8 additional finite checks passed
CSV reproduction	All 12 files byte identical to the archived outputs
Reference implementation	Commit `eb96b984ae999ee6cbc0aae33c2d501c46d9540f`

D.2 Reproduce the finite checks

Run the following commands from the `paper/` directory to check the finite models and compare all twelve CSV files.

```
python -m pytest verification/reference_code/simulations/tests \
  verification/test_scope_cases.py verification/test_round_two.py -q
python verification/verify_outputs.py
```

The two tests in `test_round_two.py` enumerate small reachable sets and pairs of binary measurements.